GEM521 Lab 4 Individual tree detection using lidR

Ellen Wu

2026-02-13

Lab Objectives

Explore `lidR` functions related to individual tree segmentation

Deliverables

- R script with code for each step
- PDF containing screenshots and answers to questions

Introduction

In this lab we will explore the process of individual tree segmentation in `lidR`. We will work with the 20 `.las` tiles for Malcolm Knapp Research Forest (MKRF) used in labs 1 and 2.

You have also been supplied an additional file: *(Lab4_Plots.csv)*.

There are two general methods used to detect individual trees from lidar data:

- **Point Cloud:** Segmentation is based solely on the normalized point cloud data Canopy
- **Height Model (CHM):** segmentation based on information from a 2D CHM

Lab Steps

Plot Extraction

Load the required packages

```r
# Load the packages needed for the lab
library(lidR)
library(terra)
```

```
## Warning: package 'terra' was built under R version 4.5.1
```

```
## terra 1.8.75
```

```
##
## Attaching package: 'terra'

## The following object is masked from 'package:lidR':
##
##     watershed
```

```r
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.5.1

## Warning: package 'ggplot2' was built under R version 4.5.2

## Warning: package 'tibble' was built under R version 4.5.1

## Warning: package 'tidyr' was built under R version 4.5.1

## Warning: package 'readr' was built under R version 4.5.1

## Warning: package 'purrr' was built under R version 4.5.1

## Warning: package 'dplyr' was built under R version 4.5.1

## Warning: package 'forcats' was built under R version 4.5.1

## Warning: package 'lubridate' was built under R version 4.5.1

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.2
## v ggplot2   4.0.1     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x tidyr::extract() masks terra::extract()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

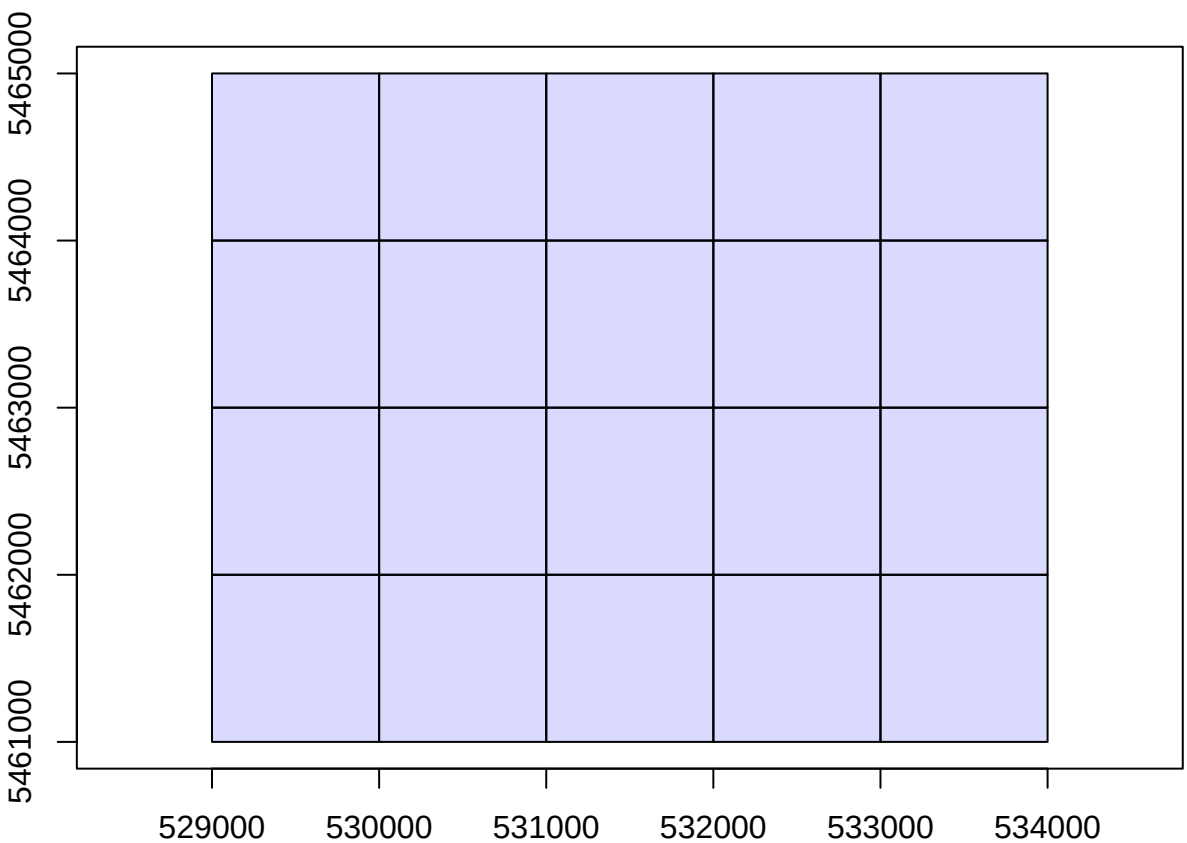
1. We have four plot locations for this lab, which are listed in the `Lab4_Plots.csv` file. Start by extracting point clouds for each plot location, using a diameter of 154 m. Use the R scripts we developed in previous labs and remove outliers below 0 m and above 65 m. Be sure to use proper naming convention in order to avoid confusion later in the lab. At this point, it is a good idea to explore each plot.

Pre-processing

```r
# Read in the Lab4_Plots.csv
plot_table <- read.csv("data/Lab4_Plots.csv")

# Read in the .las tiles
cat_mkrf <- readLAScatalog("data/LAS")
las_check(cat_mkrf)
```

Extract the 4 plots

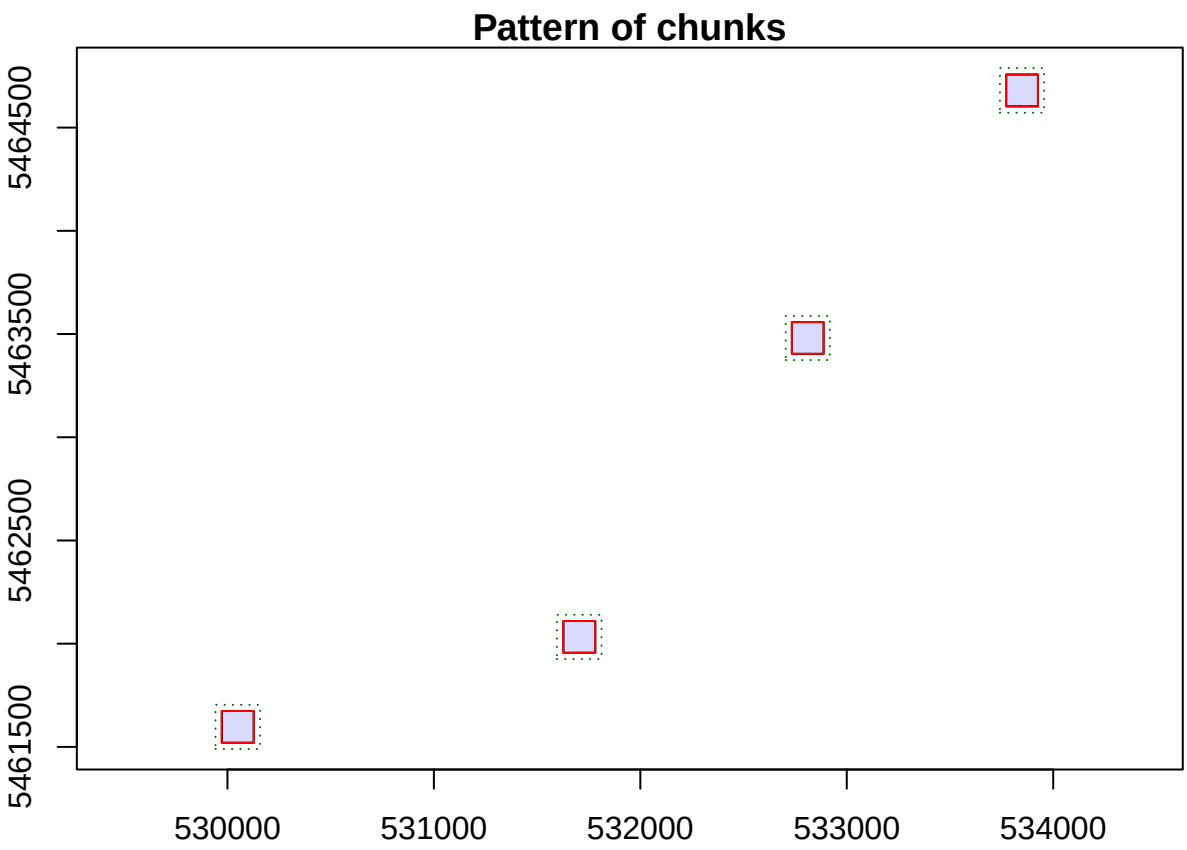
```r
## Define plot radius
radius <- 77

## For loop to extract multiple plots and write them into a folder
for(i in 1:nrow(plot_table)){ # run the loop until i = the number of rows
  plot_cent <- c(plot_table$X[i], plot_table$Y[i]) # extract plot center in X and Y
  plot_las <- clip_circle(cat_mkrf, plot_cent[1], plot_cent[2], radius)
  # clip plot from cat_mkrf
  output_file <- paste("data/Plots/Plot_", i, ".las", sep = "") # output directory
  writeLAS(assign(paste("Plot_", i, sep = ""), plot_las), output_file)
  # write'Plot_i' to output dir.
}
```



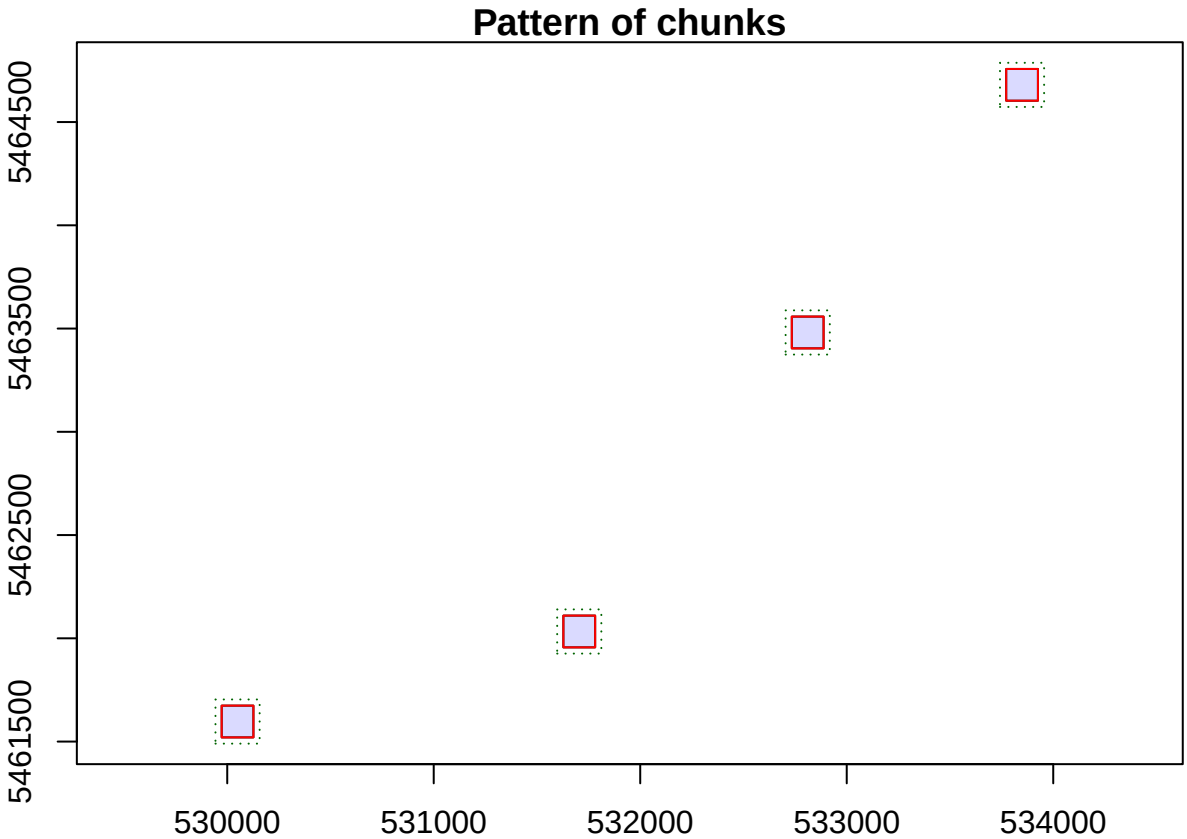

```
## Warning: package 'future' was built under R version 4.5.1
```

Normalize the 4 plots.las

```
# Read in the Plot_i.las tiles
cat_plots <- readLAScatalog("data/Plots")
las_check(cat_plots)

# Create the DEM with 2 m spatial resolution
dem_plots <- rasterize_terrain(cat_plots, 2, tin())
```




```
# Set normalized option
opt_output_files(cat_plots) <- paste("data", "/Normalized/norm_Plot_{ID}", sep = "")

# Run normalization on the whole catalog
norm_cat_plots <- normalize_height(cat_plots, dem_plots)
```




```
## Warning: 10 points do not belong in the raster. Nearest neighbor was used to
## assign a value.

## Warning: 11 points do not belong in the raster. Nearest neighbor was used to
## assign a value.

## Warning: 236 points do not belong in the raster. Nearest neighbor was used to
## assign a value.
```

```r
# Remove the outliers below 0 m and above 65 m
opt_filter(norm_cat_plots) <- "-drop_z_below 0 -drop_z_above 65"
```

Explore the four plots separately

```r
# Read in the .las separately
norm_plot1 <- readLAS("data/Normalized/norm_Plot_1.las")
norm_plot2 <- readLAS("data/Normalized/norm_Plot_2.las")
norm_plot3 <- readLAS("data/Normalized/norm_Plot_3.las")
norm_plot4 <- readLAS("data/Normalized/norm_Plot_4.las")


# Plot them in 3D and explore their structures
plot(norm_plot1) # Clearcut with around 20 trees left
```

```r
plot(norm_plot2)
# Maybe selected cut, not as full as plot3 and plot4 but with some understory veg
plot(norm_plot3) # Fully loaded with trees and many understory
plot(norm_plot4) # Dense forest but less understory than plot3
```

Detect Individual Trees using a Point Cloud

For the first part of this lab, we will detect individual trees using only the lidar point cloud data. To start, we will focus on Plot 1. This plot is located in a clearcut, where a number of trees were left standing. The crowns of these trees are clearly separated, so this plot is a good place to test our individual tree detection (i.e., we can compare detected trees against those we can see in the plot).

2. To detect individual trees, we will use the `segment_trees` function. There are multiple algorithms available in `segment_trees` and we recommend you explore each. For this section of the lab, we will use the `li2012` algorithm. Please review the Li et al., 2012 paper to understand how this algorithm works. If this link is broken, please refer to the manuscript provided to you as part of the course readings.

Question 1: In what type of forest stand do Li et al., 2012 suggest their algorithm is successful? Do the four plots you extracted match this description?

> The algorithm is successful in complex mixed conifer forests on rugged terrain. Plot 3 and Plot 4 match this description, but Plot 1 and Plot 2 does not, as trees in these two plots are sparcely distributed with minimal understory vegetation.

Question 2: Which part of the tree do Li et al., 2012 begin their point classification with? Why do they do this?

> The algorithm begins point classification from a tree top. As explained in the paper, this approach is used because the spacing between points at the top of a tree is larger, making it easier to identify and include nearby points belonging to the same tree while excluding points from neighboring trees based on their relative spacing. Starting at lower levels would make separation much more difficult, as points from adjacent trees are more likely to overlap.

To run, this function requires multiple arguments.

The first arguments are the adaptive thresholds, or `dt1` and `dt2`. These thresholds determine the distance that one point can be from another to be included in the same tree. They are considered adaptive because the algorithm chooses to use `dt1` or `dt2` based on the height of the seed point, which is a technical term for tree top. To be accurate, it is necessary to determine the spacing of trees within the area being classified. This can cause challenges because it is difficult to calculate tree spacing without first accurately segmenting individual trees.

The argument `R` refers to the search radius for points around the local maxima, which has been determined by Li et al., 2012 to be 2 m in most cases.

The argument `Zu` determines which threshold is used by evaluating the height of the seed point. If `Zu = 15`, then any seed point higher than 15 m will use `dt2`. Any seed point lower than 15 m will use `dt1`. Li et al., 2012 have determined default values through trial and error: `dt1 = 1.5`, `dt2 = 2`, `Zu = 15`, and `R = 2`.

For this lab, we will use the default values determined by Li et al., 2012 for all arguments: `dt1 = 1.5`, `dt2 = 2`, `R = 2`, `Zu = 15` and `speed_up = 10`.

We will also use the default value for `hmin`, which is 2 m, as it follows with previous labs in excluding points below 2 m from our analyses.

Tree segmentation with raw point clouds

```
# Run the segment_trees function to detect individual tree in each plot
plot1_trees <- segment_trees(norm_plot1, li2012())
plot2_trees <- segment_trees(norm_plot2, li2012())
plot3_trees <- segment_trees(norm_plot3, li2012())
plot4_trees <- segment_trees(norm_plot4, li2012())

# Plot the segmentation results for each plot
plot(plot1_trees, color = "treeID")
plot(plot2_trees, color = "treeID")
plot(plot3_trees, color = "treeID")
plot(plot4_trees, color = "treeID")
```

Question 3: Define the `speed_up` argument in the `li2012` algorithm.

> The `speed_up` argument, as described in the algorithm's documentation, defines the maximum radius of a tree crown. This parameter does not affect the segmentation results but influences computational efficiency by limiting the number of point-to-point comparisons performed during processing. Therefore, if the maximum crown radius is known or can be reasonably estimated based on the forest type and tree species, this parameter can be manually set to significantly reduce processing time without compromising segmentation accuracy.

Screenshot 1: Include a figure of the output from segmenting each plot with `li2012` colored by Tree ID.

> See Figure 1 - Figure 4

Although it is not mandatory for successful segmentation, it can be beneficial to manually determine some arguments, such as `speed_up`, for the specific study area. For example, imagine that you are working with lidar data for a young tree plantation. The trees are still relatively small, with the largest tree crown being 4 m in diameter (2 m radius), and the crowns are very close together. In this case, the default `speed_up` value of 10 is five times larger than it needs to be based on the crown radius, which could cause multiple issues. For example, the close crowns and oversized `speed_up` value could cause multiple trees to be classified as one. Depending on the size of your study area and `.las` file, it is also possible that this `speed_up` value could affect the processing speed of your segmentation. For the sake of accuracy and time, it can be useful to manually estimate the maximum crown width present on your site.

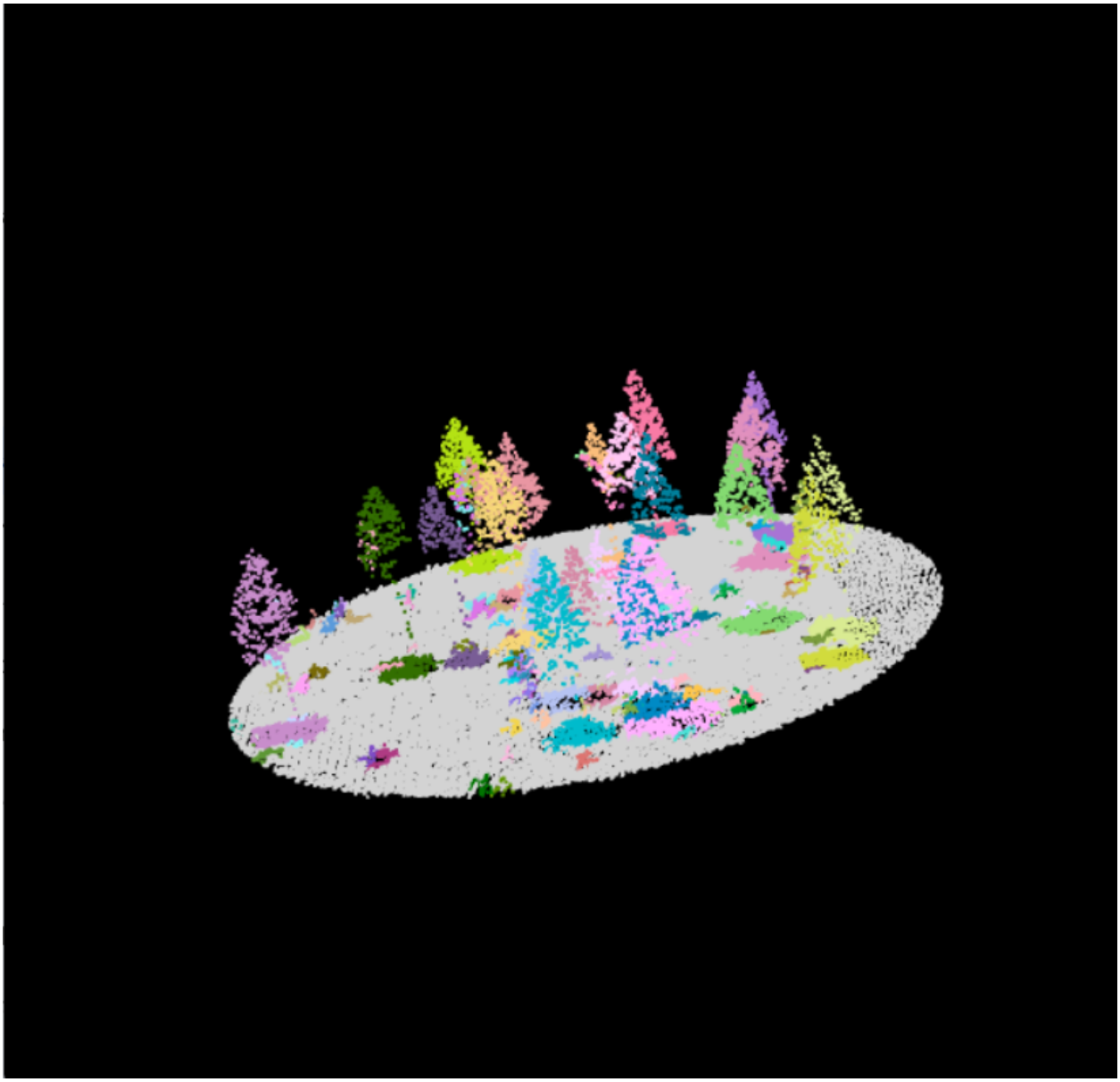


Figure 1: Plot 1

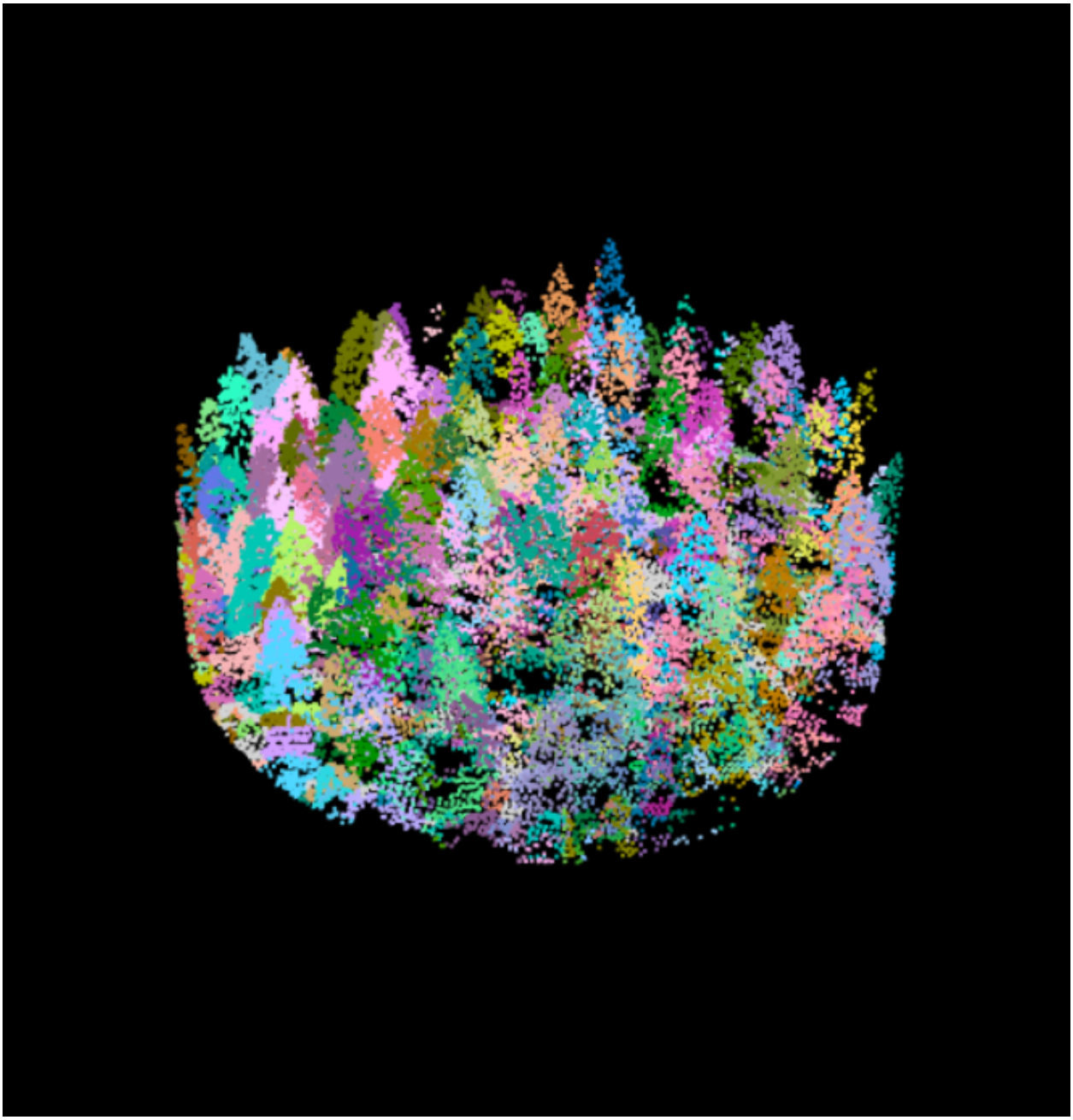


Figure 2: Plot 2

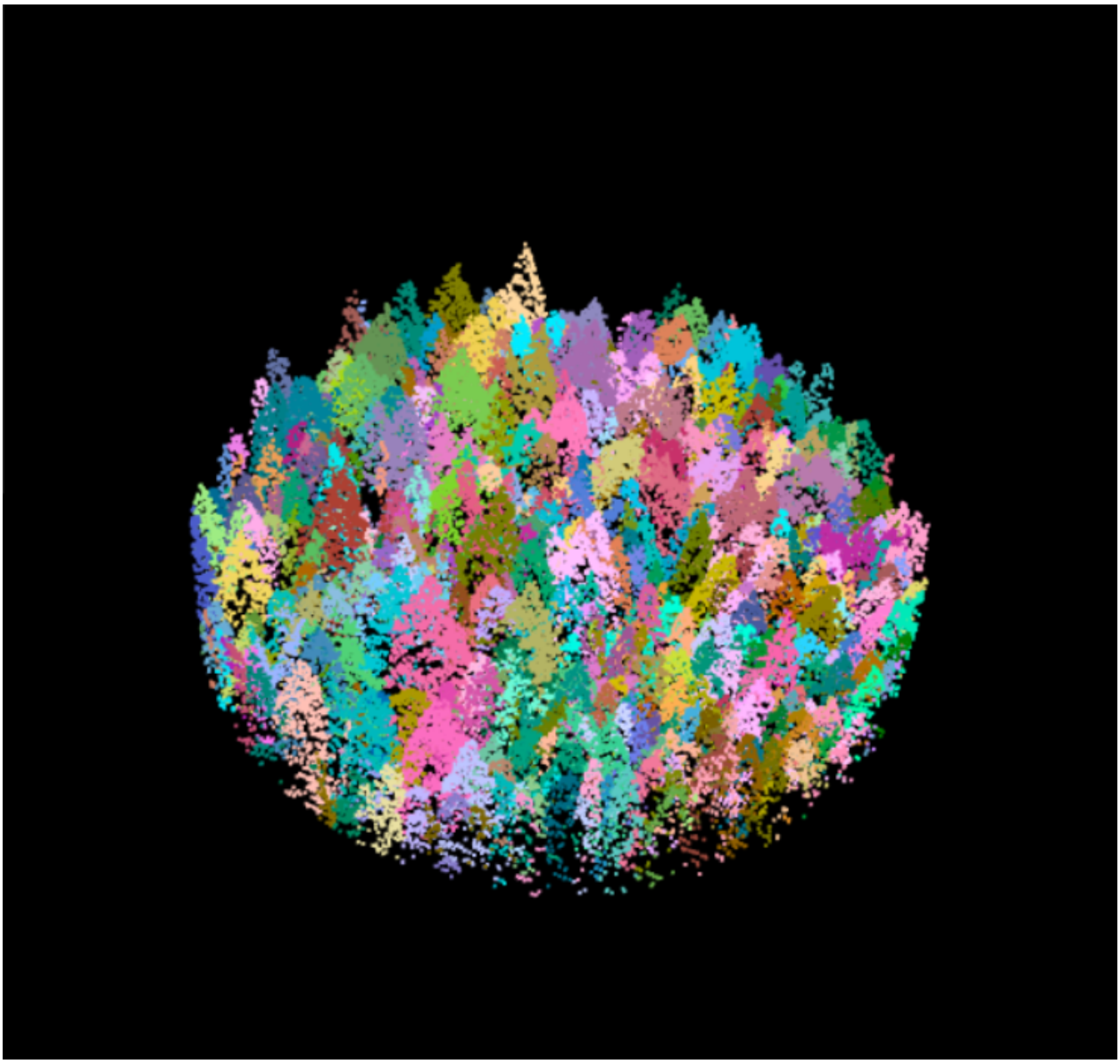


Figure 3: Plot 3

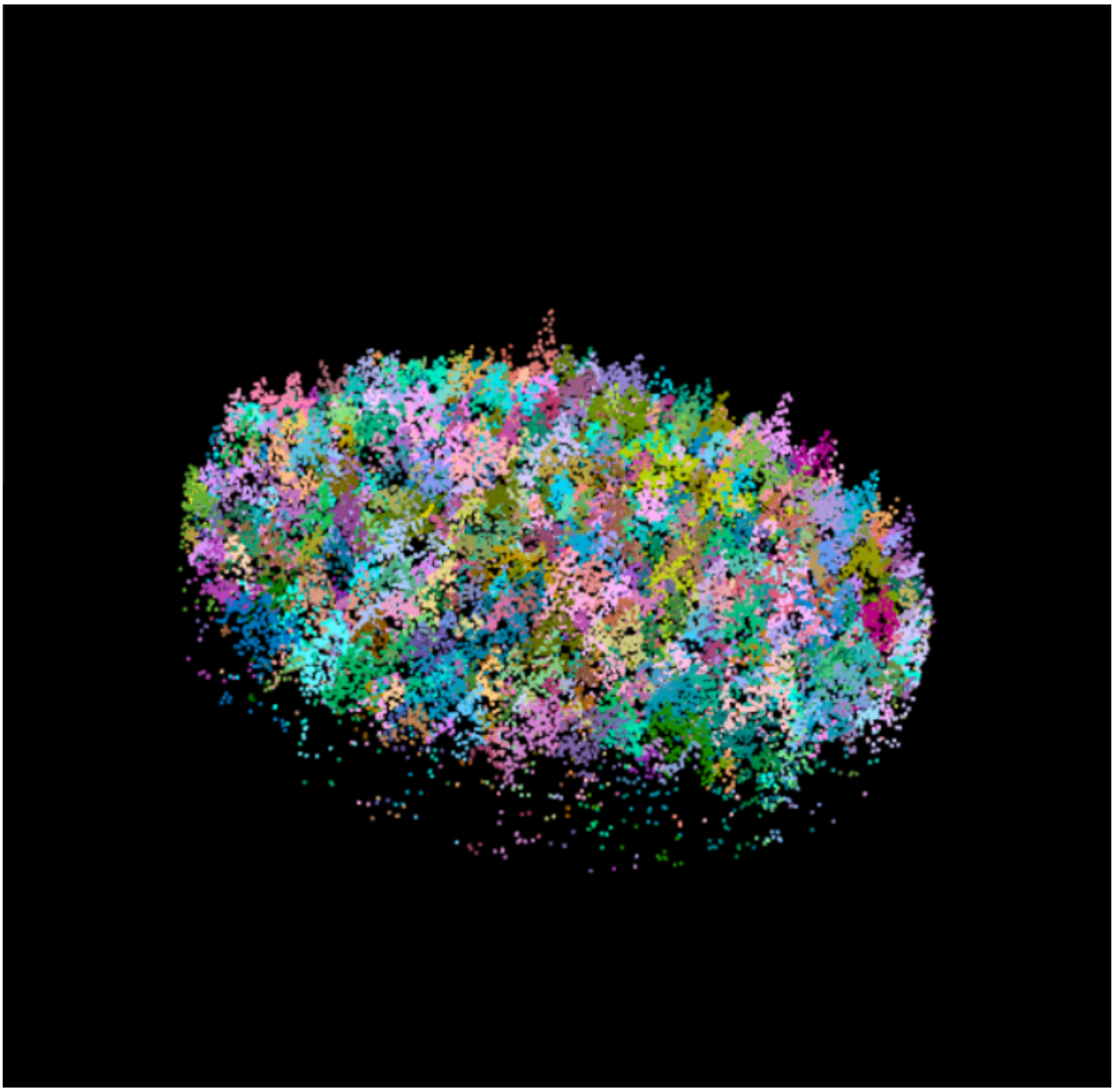


Figure 4: Plot 4

Detect Individual Trees using a CHM

In the previous section of this lab we used raw point cloud information to detect individual trees. Now, we will use a CHM in an attempt to increase classification accuracy. There are a variety of CHM-based segmentation algorithms within lidR, but we will use the one presented by Dalponte and Coomes 2016.

Question 4: What is a key advantage of the tree-centric approach over existing area-based methods according to Dalponte and Coomes 2016?

> According to Dalponte and Coomes (2016), the tree-centric approach can "produce maps at any scale and is fundamentally based on field based inventory methods, making it intuitive and transparent".

---

3. We will use the Dalponte and Coomes 2016 algorithm. Explore the `Help` page for this algorithm to understand it's details. You will notice that the arguments used in `li2012` are different to those used in `dalponte2016`:

`chm` refers to a `RasterLayer` canopy height model. `treetops` refers to the spatial location of individual trees determined by `locate_trees` or an external shapefile `th_tree` is the threshold below which a pixel cannot be a tree (default = 2) `th_seed` is Growing Threshold 1 (default is 0.45) `th_c` is the Growing Threshold 2 (default is 0.55) `max_cr` refers to the maximum value of the crown diameter of a detected tree (default = 10) `ID` individual identification number of each tree.

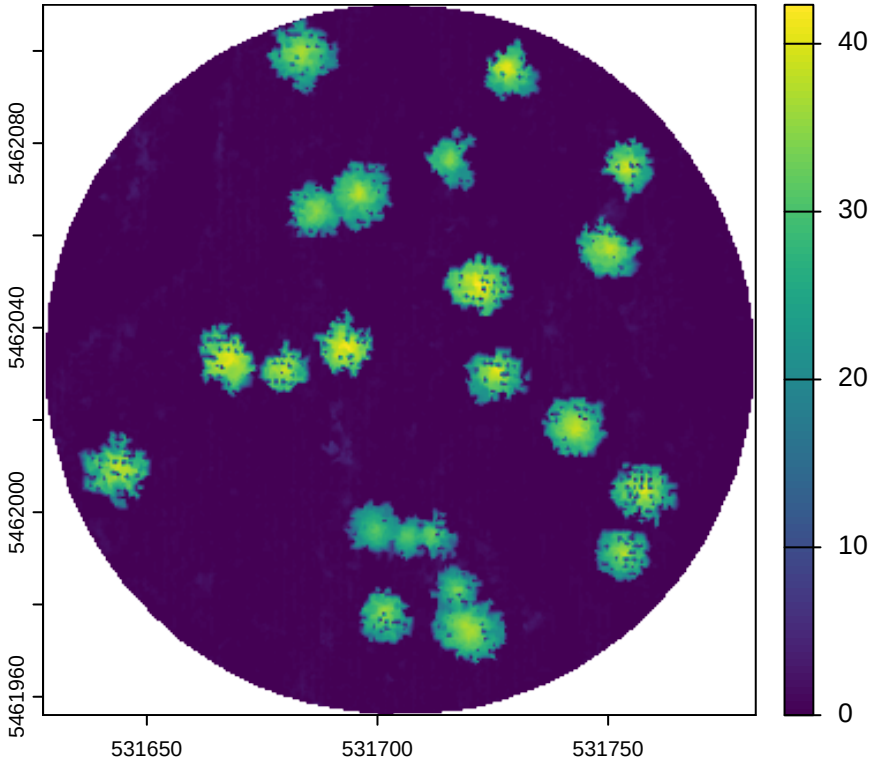
Question 5: Explain the difference between the th_seed and th_cr arguments of the dalponte2016 algorithm.

> "th_seed" uses a fixed threshold based on the tree's absolute maximum height66, while "th_cr" uses a dynamic threshold based on the average height of the pixels already assigned to that specific region.

---

4. The first step in detecting individual trees using a CHM is, unsurprisingly, creating a CHM. For this step we will work with **Plot 1** at a spatial resolution of 0.5 m. To do this, we will use the `pitfree` algorithm with height thresholds at 0 m, 10 m, 20 m and 30 m and a sub-circle with a 20 cm radius. The maximum edge length should be 1 with the classical triangulation threshold = 0. Plot the CHM to ensure it is `pitfree`.

```r
# Create a CHM for plot 1 at a spatial resolution of 0.5 m
chm_plot1 <- rasterize_canopy(norm_plot1, 0.5,
                              pitfree(thresholds = c(0, 10, 20, 30),
                                      max_edge = c(0,1),
                                      subcircle = 0.2))

# Plot the plot 1 CHM
plot(chm_plot1)
```



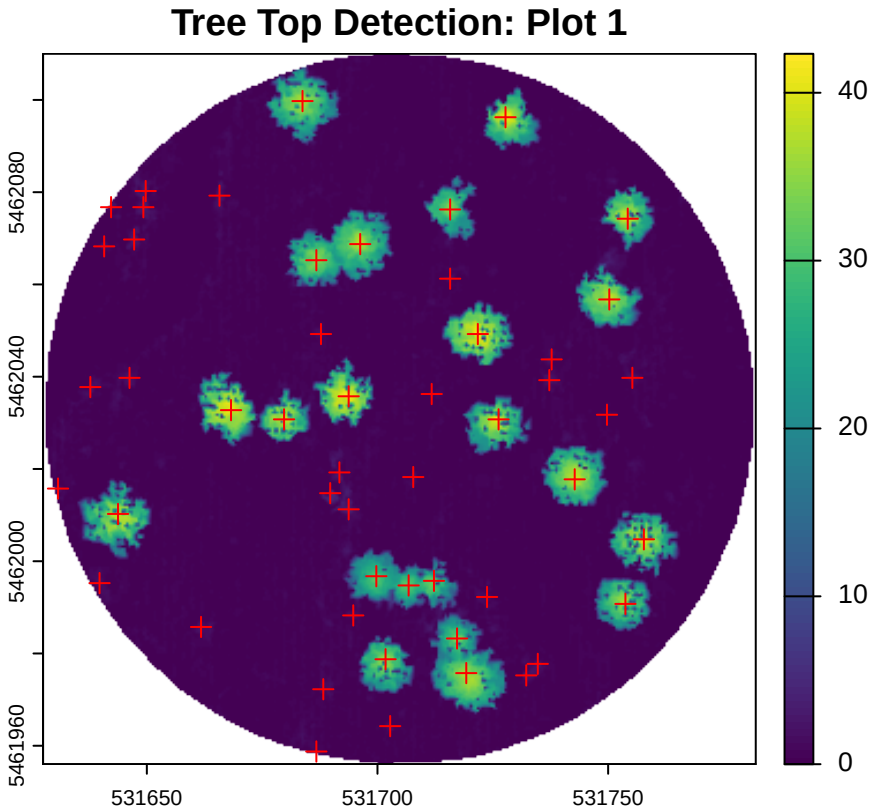
5. Now we need to determine the tree tops for Plot 1. Use `locate_trees` with the `lmf` algorithm specifying a circular moving window of 5 m and minimum tree height of 2 m.

```
# Determine the tree tops for Plot 1
treetop_plot1 <- locate_trees(chm_plot1, lmf(ws = 5,  hmin = 2, shape = "circular"))

# Tree segmentation for Plot 1
seg_plot1 <- segment_trees(norm_plot1, dalponte2016(chm_plot1, treetop_plot1))
```

Screenshot 2: Include a figure of your Plot 1 CHM with the tree tops displayed (HINT: use `add = TRUE` when plotting tree tops. Ensure the previous plot was your CHM).

```
# Plot 1 CHM with the tree tops displayed as cross
plot(chm_plot1, main = "Tree Top Detection: Plot 1")
plot(treetop_plot1, add = TRUE, col = "red", pch = 3)
```

```
## Warning in plot.sf(treetop_plot1, add = TRUE, col = "red", pch = 3): ignoring
## all but the first attribute
```



---

6. Working with Plot 2, complete segmentation at 0.5 m, 2 m, 4 m and 10 m spatial resolution

Tree segmentation at 0.5 m

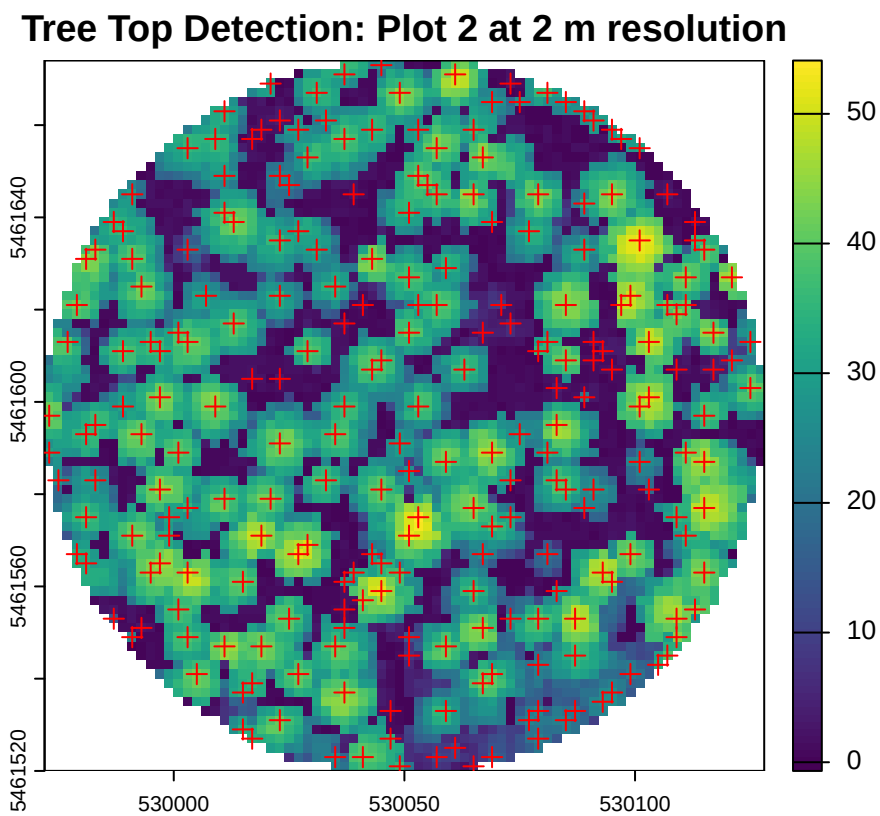
```
# CHM at 0.5 m
chm_plot2_05 <- rasterize_canopy(norm_plot2, 0.5, p2r())

# Treetop for Plot 2 at 0.5 m
treetop_plot2_05 <- locate_trees(chm_plot2_05, lmf(ws = 5,  hmin = 2,
                                                   shape = "circular"))

# Tree segmentation at 0.5 m
seg_plot2_05 <- segment_trees(norm_plot2,
                              dalponte2016(chm_plot2_05, treetop_plot2_05))
```

Tree segmentation at 2 m

```
# CHM at 2 m
chm_plot2_2 <- rasterize_canopy(norm_plot2, 2, p2r())
```

```r
# Treetop for Plot 2 at 2 m
treetop_plot2_2 <- locate_trees(chm_plot2_2, lmf(ws = 5,  hmin = 2,
                                                 shape = "circular"))

# Tree segmentation at 2 m
seg_plot2_2 <- segment_trees(norm_plot2,
                             dalponte2016(chm_plot2_2, treetop_plot2_2))

# Plot the CHM with treetop
plot(chm_plot2_2, main = "Tree Top Detection: Plot 2 at 2 m resolution")
plot(treetop_plot2_2, add = TRUE, col = "red", pch = 3)
```

```
## Warning in plot.sf(treetop_plot2_2, add = TRUE, col = "red", pch = 3): ignoring
## all but the first attribute
```



Tree segmentation at 4 m

```r
# CHM at 4 m
chm_plot2_4 <- rasterize_canopy(norm_plot2, 4, p2r())

# Treetop for Plot 2 at 4 m
treetop_plot2_4 <- locate_trees(chm_plot2_4, lmf(ws = 5,  hmin = 2,
```

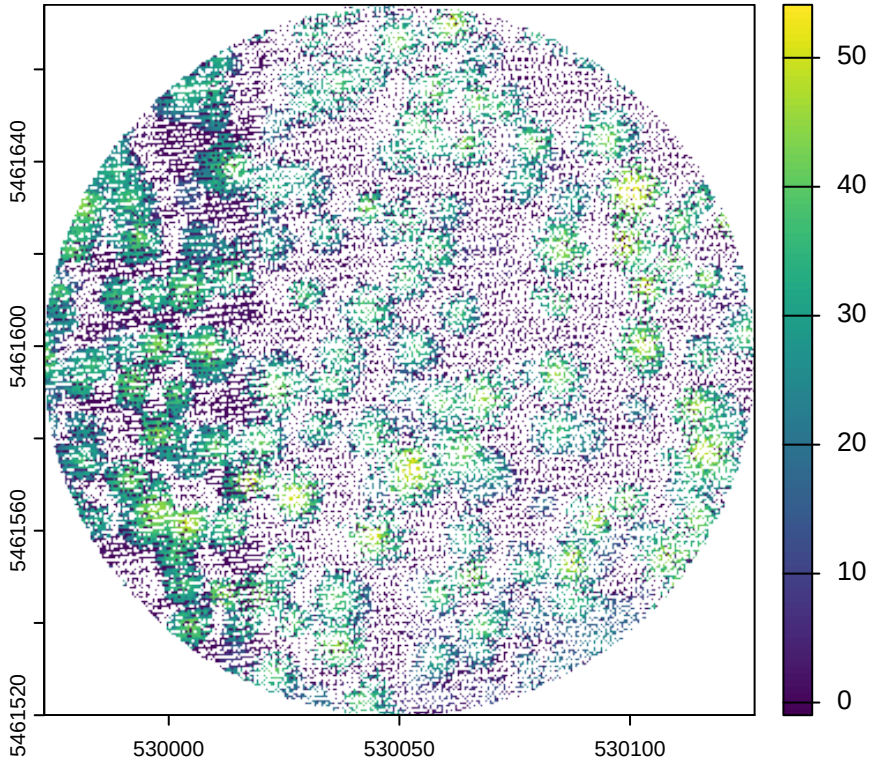
```r
                                                shape = "circular"))

# Tree segmentation at 4 m
seg_plot2_4 <- segment_trees(norm_plot2,
                             dalponte2016(chm_plot2_4, treetop_plot2_4))
```

Tree segmentation at 10 m

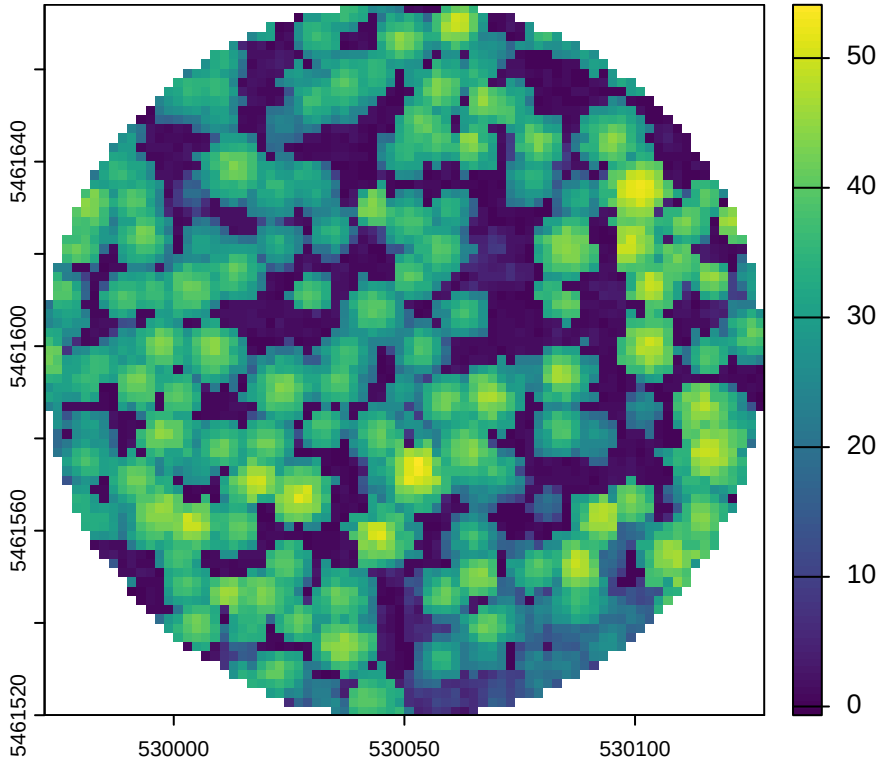
```r
# CHM at 10 m
chm_plot2_10 <- rasterize_canopy(norm_plot2, 10, p2r())

# Treetop for Plot 2 at 10 m
treetop_plot2_10 <- locate_trees(chm_plot2_10, lmf(ws = 5,  hmin = 2,
                                                shape = "circular"))

# Tree segmentation at 10 m
seg_plot2_10 <- segment_trees(norm_plot2,
                              dalponte2016(chm_plot2_10, treetop_plot2_10))
```

Screenshot 3: Include a figure of the four CHMs created in Step 6.

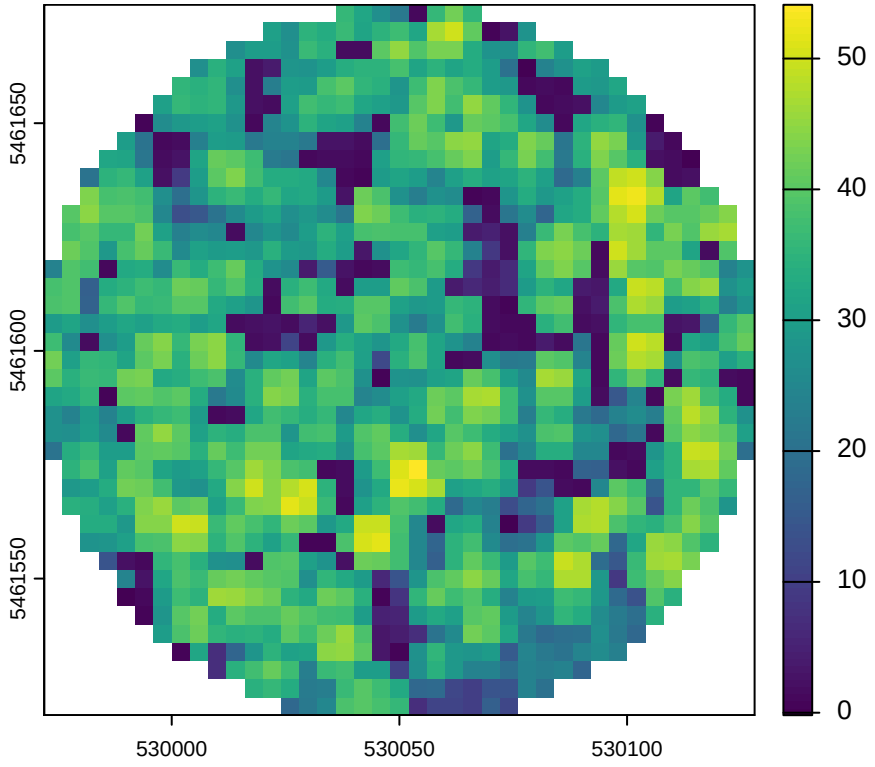
```r
# Plot the plot 2 in different resolution
plot(chm_plot2_05)
```



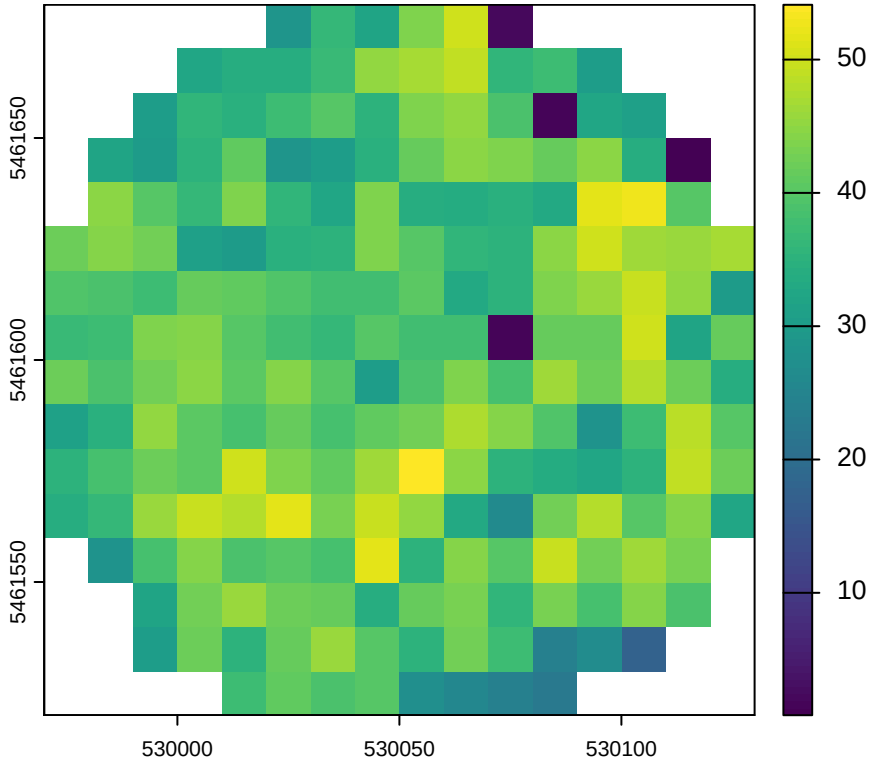

```r
plot(chm_plot2_2)
```




```r
plot(chm_plot2_4)
```




```
plot(chm_plot2_10)
```



Screenshot 4: Include a figure of ONE rgl (3D) plot created in Step 6 (with dalponte).

```
# Plot the 3D plot at all resolutions
plot(seg_plot2_05, color = "treeID")
plot(seg_plot2_2, color = "treeID")
plot(seg_plot2_4, color = "treeID")
plot(seg_plot2_10, color = "treeID")
```

See Figure 5

Question 6: If you had to select a spatial resolution to conduct segmentation at for the entire MKRF, what would it be? Justify your answer.

I will select the 2 m spatial resolution for the entire MKRF. Because finer resolutions (0.5 m) capture small gaps between branches, leaving many points unassigned to any tree ID (i.e. many "grey trees" that were unassigned to any tree ID). Coarse resolutions (4 m and 10 m) blur multiple tree tops into a single pixel or create artificial plateaus, incorrectly splitting a single tree into multiple IDs (i.e. shaded a single tree with multiple colors in the 3D plots).

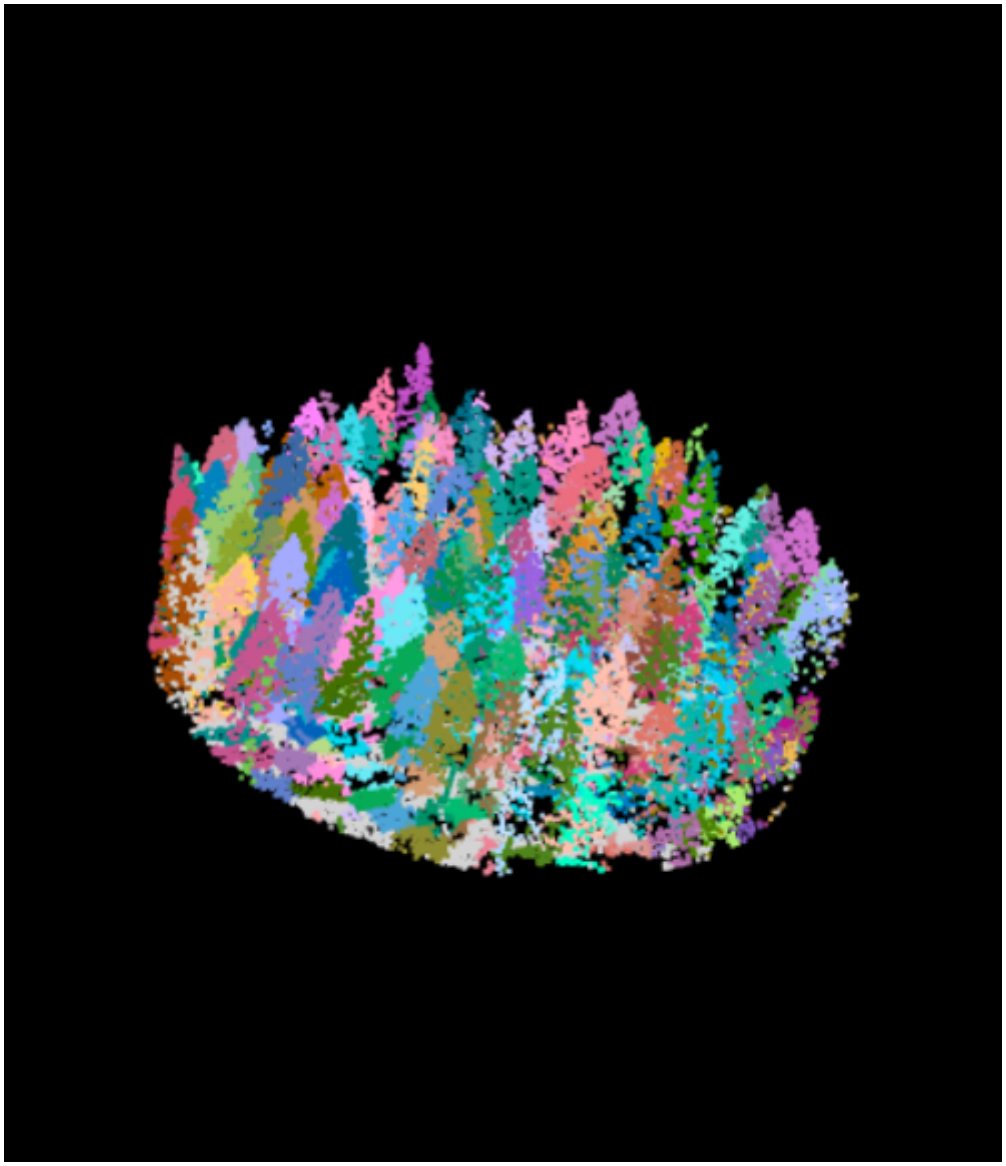


Figure 5: 3D plot for Plot 2 at 2 m resolution